Module – 2:
Graph Connectivity

Dr. Nirnay Ghosh

Connectivity

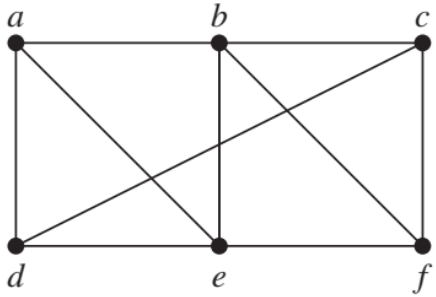
- Path:

Let n be a nonnegative integer and G an undirected graph. A *path* of *length* n from u to v in G is a sequence of n edges $e_1, \ldots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \ldots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \ldots, n$, the endpoints x_{i-1} and x_i. When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \ldots, x_n$ (because listing these vertices uniquely determines the path). The path is a *circuit* if it begins and ends at the same vertex, that is, if $u = v$, and has length greater than zero. The path or circuit is said to *pass through* the vertices $x_1, x_2, \ldots, x_{n-1}$ or *traverse* the edges $e_1, e_2, \ldots, e_n$. A path or circuit is *simple* if it does not contain the same edge more than once.

- Informally, a path is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex along edges of the graph.
- As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these edges.

Connectivity

Let n be a nonnegative integer and G a directed graph. A *path* of length n from u to v in G is a sequence of edges $e_1, e_2, \ldots, e_n$ of G such that e_1 is associated with (x_0, x_1), e_2 is associated with (x_1, x_2), and so on, with e_n associated with (x_{n-1}, x_n), where $x_0 = u$ and $x_n = v$. When there are no multiple edges in the directed graph, this path is denoted by its vertex sequence $x_0, x_1, x_2, \ldots, x_n$. A path of length greater than zero that begins and ends at the same vertex is called a *circuit* or *cycle*. A path or circuit is called *simple* if it does not contain the same edge more than once.



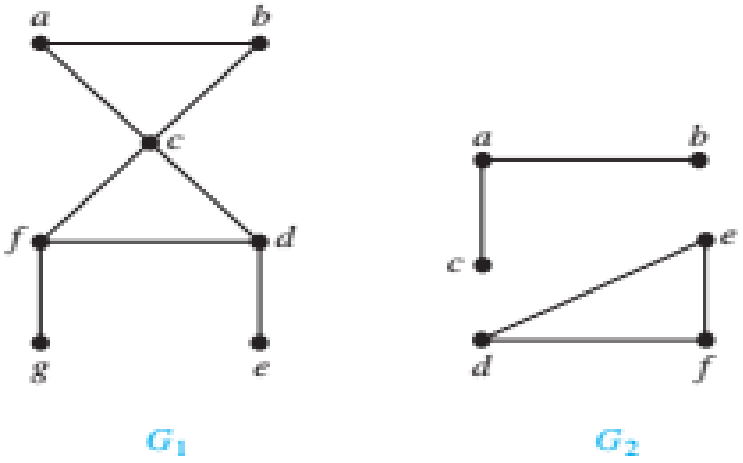
A Simple Graph

- a, d, c, f , e is a simple path of length 4, because {a, d}, {d, c}, {c, f }, and {f, e} are all edges. However, d, e, c, a is not a path, because {e, c} is not an edge.
- b, c, f , e, b is a circuit of length 4 because {b, c}, {c, f }, {f, e}, and {e, b} are edges, and this path begins and ends at b.
- The path a, b, e, d, a, b, which is of length 5, is not simple because it contains the edge {a, b} twice.

Connectivity

- Connectedness in Undirected Graphs

An undirected graph is called *connected* if there is a path between every pair of distinct vertices of the graph. An undirected graph that is not *connected* is called *disconnected*. We say that we *disconnect* a graph when we remove vertices or edges, or both, to produce a disconnected subgraph.



Connected and Disconnected Graph

- Theorem:

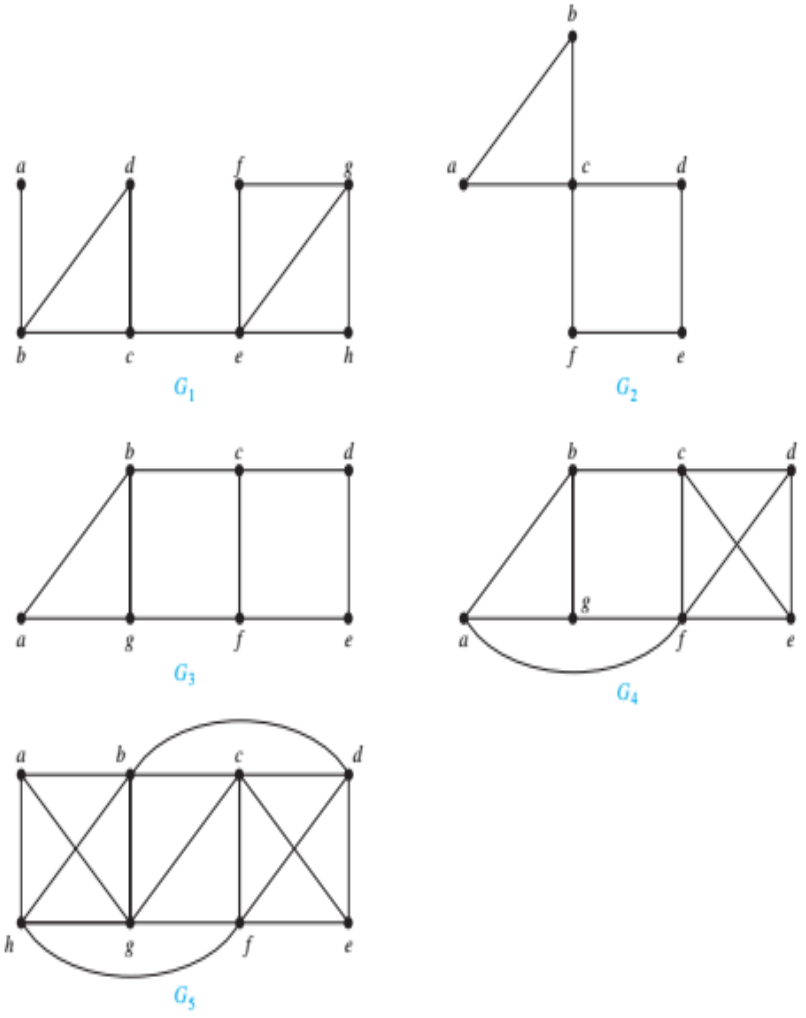
There is a simple path between every pair of distinct vertices of a connected undirected graph.

Connectivity: How connected is a graph?

- **Cut vertex (articulation point)**: removal of a cut vertex from a connected graph produces a subgraph that is not connected.
  - Complete graph K_n, where $n \geq 3$, has no cut vertex.
  - Connected graphs without cut vertices are called *inseparable graphs*
- **Cut edge or bridge**: an edge whose removal produces a graph with more connected components than in the original graph.
  - Example: In a computer network, a cut vertex is an essential router and cut edge is an essential link that cannot fail for a computer network to be operational.
- If *G* does not have a cut vertex, we look for the *smallest set of vertices* that can be removed to disconnect it.
- **Vertex cut**: A subset *V'* of the vertex set *V* of $G = (V, E)$ is a *vertex cut, or separating set*, if *G - V'* is disconnected.
  - ***Vertex connectivity*** of a noncomplete graph *G*, denoted by $\kappa(G)$, is the minimum number of vertices in a ***vertex cut (since, there may be multiple vertex cuts)***

Connectivity

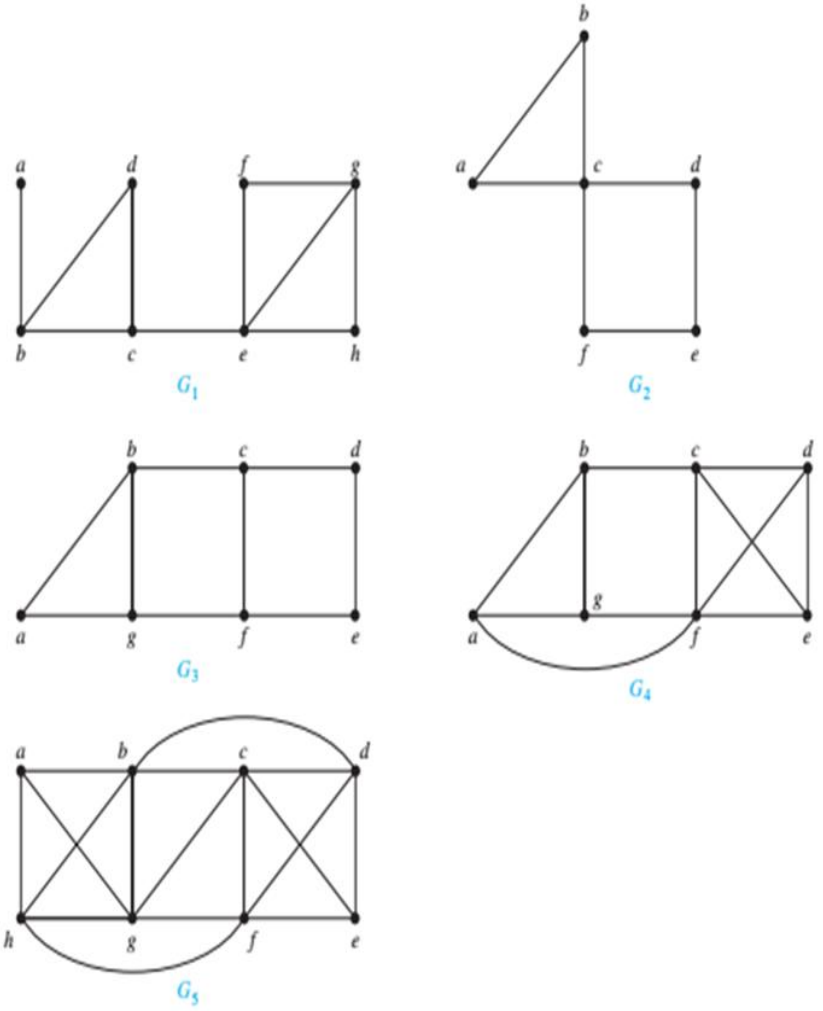
- In simple term, *vertex connectivity* of a noncomplete graph G, denoted by $\kappa(G)$, is the minimum number of vertices that can be removed from G to either disconnect G or produce a graph with a single vertex
- The larger $\kappa(G)$ is, the more connected G to be
- 1-connected: connected (has path between every pair of vertices), $\kappa(G) \geq 1$
- 2-connected:connected but with no cut vertex (articulation point), $\kappa(G) \geq 2$
  - Removing any single vertex does not disconnect the graph
- k-connected: a graph is k-connected (or k-vertex-connected), if $\kappa(G) \geq k$.
- If G is a k-connected graph, then G is also j-connected graph for all j with $0 \leq j \leq k$.



Find the vertex connectivity of the given graphs.

Connectivity

- If a graph has a ***cut edge***, then we need to remove it only to disconnect *G*.
- If *G* does not have a *cut edge*, we look for the *smallest* set of edges that can be removed to disconnect it.
- **Edge cut**: A set of edges *E'* is called an *edge cut* of *G* if the subgraph *G - E'* is disconnected.
- **Edge connectivity**: The edge connectivity of a graph *G*, denoted by $\lambda(G)$, is the minimum number of edges in an edge cut of *G (since, there may be multiple edge cuts.)*



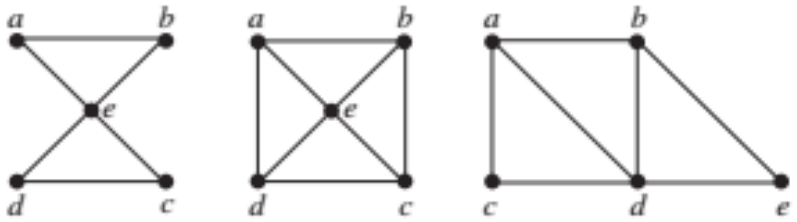
Find the edge connectivity of the given graphs.

Euler Circuits & Paths

- Euler Circuit and Path:
  - Travel along the edges of a graph starting at a vertex and returning to it by traversing each edge of the graph exactly once.

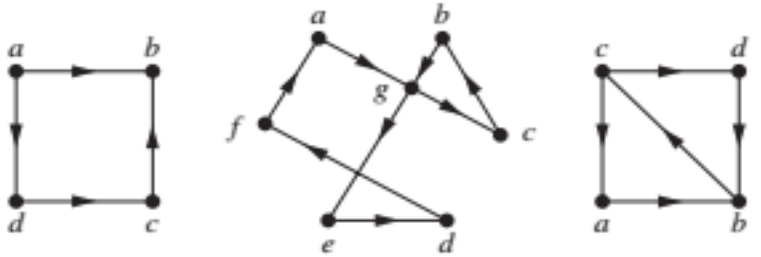
An *Euler circuit* in a graph G is a simple circuit containing every edge of G. An *Euler path* in G is a simple path containing every edge of G.

- Which of the undirected graphs have a Euler circuit? Of those that do not, which have a Euler path?



G_1 G_2 G_3

- Which of the directed graphs have a Euler circuit? Of those that do not, which have a Euler path?



H_1 H_2 H_3

Euler Circuits & Paths (*Contd...*)

- Applications of Euler paths and circuits:
  - Many applications ask for a path or circuit that traverses each street in a neighborhood, each road in a transportation network, each connection in a utility grid, or each link in a communications network exactly once.
  - Finding a Euler path or circuit in the appropriate graph model can solve such problems.
  - Example: if a postman can find a Euler path in the graph that represents the streets he needs to cover, it produces a route that traverses each street of the route exactly once.
  - If no Euler path exists, some streets will have to be traversed more than once.

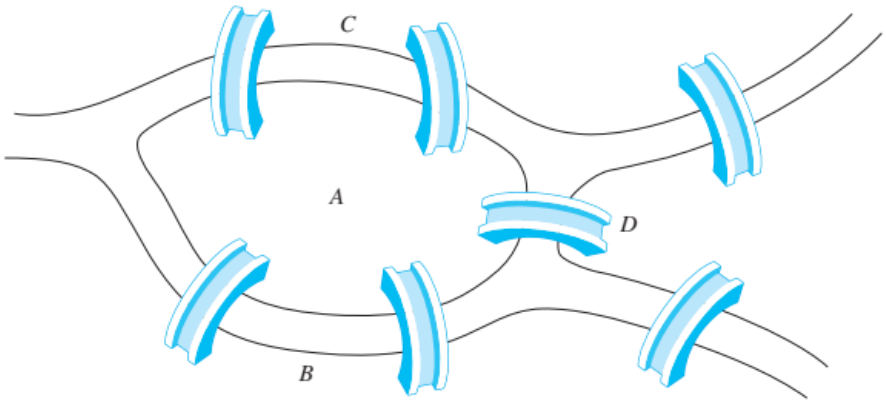


FIGURE 1 **The Seven Bridges of Königsberg.**

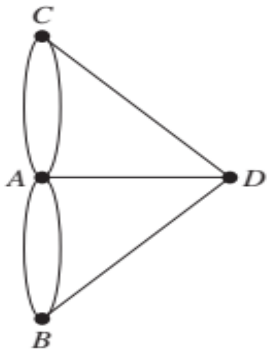


FIGURE 2 **Multigraph Model of the Town of Königsberg.**

Does this multigraph contain Euler circuits and paths?

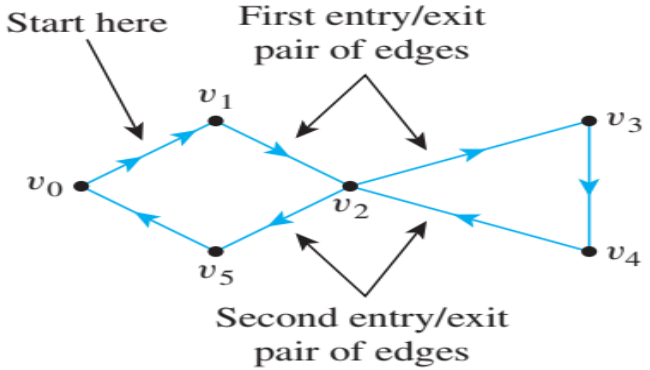
Euler Circuits & Paths (*Contd...*)

Theorem 10.2.2

If a graph has an Euler circuit, then every vertex of the graph has positive even degree.

Proof:

Suppose G is a graph that has an Euler circuit. *[We must show that given any vertex v of G, the degree of v is even.]* Let v be any particular but arbitrarily chosen vertex of G. Since the Euler circuit contains every edge of G, it contains all edges incident on v. Now imagine taking a journey that begins in the middle of one of the edges adjacent to the start of the Euler circuit and continues around the Euler circuit to end in the middle of the starting edge. (See Figure 10.2.3. There is such a starting edge because the Euler circuit has at least one edge.) Each time v is entered by traveling along one edge, it is immediately exited by traveling along another edge (since the journey ends in the *middle* of an edge).



In this example, the Euler circuit is $v_0v_1v_2v_3v_4v_5v_0$, and v is v_2. Each time v_2 is entered by one edge, it is exited by another edge.

Figure 10.2.3 Example for the Proof of Theorem 10.2.2

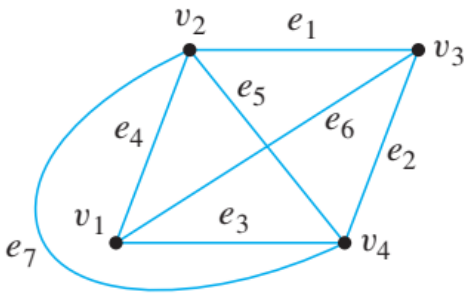
Euler Circuits & Paths (Contd...)

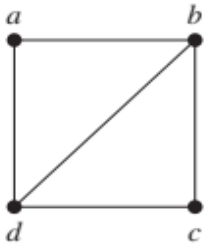
- Every vertex must have even degree
- The vertex from which the circuit begins and terminates must be of at least degree two.
- Each time the circuit passes through other vertices it contributes two to the vertices' degrees, because the circuit enters via an edge incident with a vertex and leaves via another such edge.

Euler Circuits & Paths (*Contd...*)

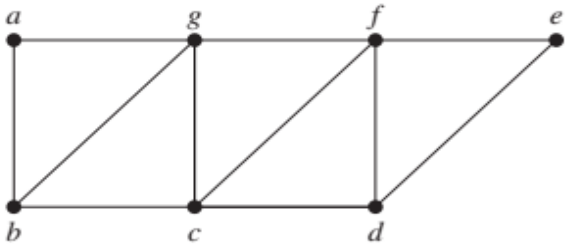
Contrapositive Version of Theorem 10.2.2

If some vertex of a graph has odd degree, then the graph does not have an Euler circuit.

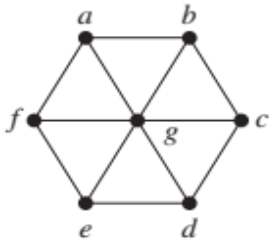




G_1

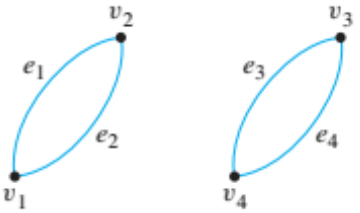


G_2



G_3

- Is the converse of the theorem true?
  - If every vertex has even degree, then the graph has a Euler circuit.



Every vertex has even degree, but the graph does not have an Euler circuit.

- Can we modify the converse by incorporating connectedness?
- ***Theorem: A connected graph with at least two vertices has a Euler circuit if and only if every vertex has even degree.***

Euler Circuits & Paths (*Contd...*)

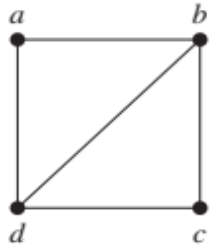
- Theorem:

A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree.

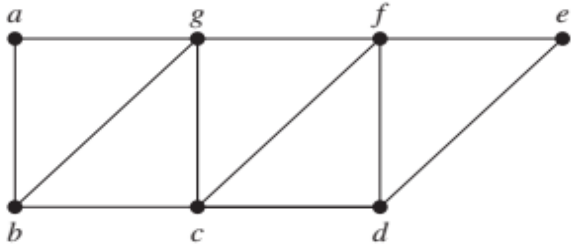
- Let a connected multigraph have a Euler path from a to b, but not a Euler circuit.
- The first edge of the path contributes one to the degree of a. A contribution of two to the degree of a is made every time the path passes through a.
- The last edge in the path contributes one to the degree of b. Every time the path goes through b there is a contribution of two to its degree.
- Consequently, both a and b have odd degree.
- Every other vertex has even degree, because the path contributes two to the degree of a vertex whenever it passes through it.
- Which graphs have a Euler path?

ALGORITHM 1 Constructing Euler Circuits.

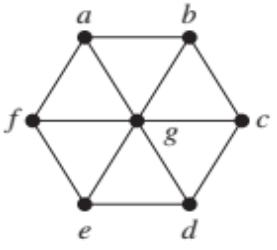
procedure *Euler*(G: connected multigraph with all vertices of even degree)
circuit := a circuit in G beginning at an arbitrarily chosen vertex with edges successively added to form a path that returns to this vertex
H := G with the edges of this circuit removed
while H has edges
  subcircuit := a circuit in H beginning at a vertex in H that also is an endpoint of an edge of *circuit*
  H := H with edges of *subcircuit* and all isolated vertices removed
  circuit := *circuit* with *subcircuit* inserted at the appropriate vertex
return *circuit* {*circuit* is an Euler circuit}



G_1



G_2



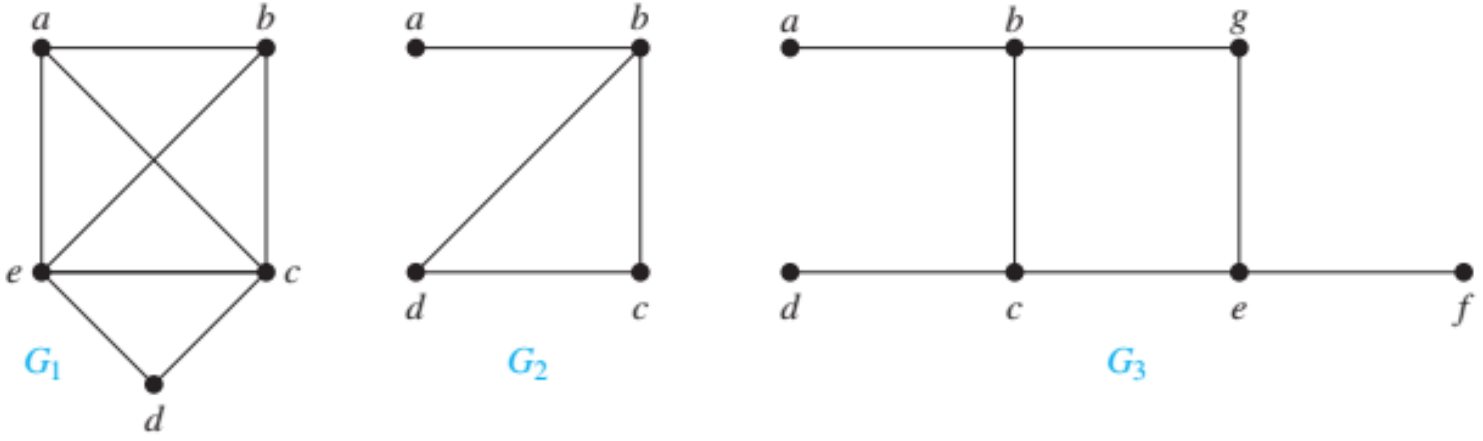
G_3

Hamilton Paths & Circuits

- Definition:

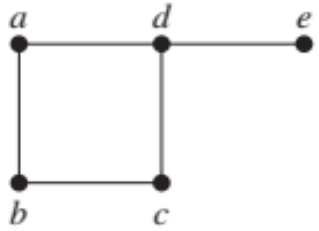
A simple path in a graph G that passes through every vertex exactly once is called a *Hamilton path*, and a simple circuit in a graph G that passes through every vertex exactly once is called a *Hamilton circuit*. That is, the simple path $x_0, x_1, \ldots, x_{n-1}, x_n$ in the graph $G = (V, E)$ is a Hamilton path if $V = \{x_0, x_1, \ldots, x_{n-1}, x_n\}$ and $x_i \neq x_j$ for $0 \leq i < j \leq n$, and the simple circuit $x_0, x_1, \ldots, x_{n-1}, x_n, x_0$ (with $n > 0$) is a Hamilton circuit if $x_0, x_1, \ldots, x_{n-1}, x_n$ is a Hamilton path.

- Which of the following simple graphs have a Hamilton circuit or, if not, a Hamilton path?

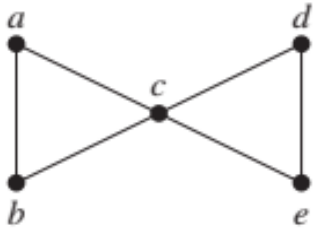


Hamilton Paths & Circuits (*Contd...*)

- Conditions for the existence of Hamilton circuits
  - There are no known simple necessary criteria for the existence of Hamilton circuits.
- Certain properties can be used to show that a graph has no Hamilton circuit.
  - A graph with a vertex of degree one cannot have a Hamilton circuit, because in a Hamilton circuit, each vertex is incident with two edges in the circuit.
  - If a vertex in the graph has degree two, then both edges that are incident with this vertex must be part of any Hamilton circuit.
  - When a Hamilton circuit is being constructed and this circuit has passed through a vertex, then all remaining edges incident with this vertex, other than the two used in the circuit, can be removed from consideration.
  - A Hamilton circuit cannot contain a smaller circuit within it.
- Show that neither graph displayed below has a Hamilton circuit.



G



H

Hamilton Paths & Circuits (*Contd...*)

- Sufficient condition for the existence of Hamilton circuits:
  - The more edges a graph has, the more likely it is to have a Hamilton circuit.
  - Adding edges (but not vertices) to a graph with a Hamilton circuit produces a graph with the same Hamilton circuit.
  - If we add edges to a graph, especially when we make sure to add edges to each vertex, we make it increasingly likely that a Hamilton circuit exists in this graph.
  - Theorems:

DIRAC'S THEOREM If G is a simple graph with n vertices with $n \geq 3$ such that the degree of every vertex in G is at least $n/2$, then G has a Hamilton circuit.

ORE'S THEOREM If G is a simple graph with n vertices with $n \geq 3$ such that $\deg(u) + \deg(v) \geq n$ for every pair of nonadjacent vertices u and v in G, then G has a Hamilton circuit.

- Both Ore's theorem and Dirac's theorem provide sufficient conditions for a connected simple graph to have a Hamilton circuit. However, these theorems do not provide necessary conditions for the existence of a Hamilton circuit.

Connected Component

- Undirected graphs: can have disjoint connected subgraphs (components).
- A connected component of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G.
- It is a maximal connected subgraph of G.
  - "Maximal" means you cannot add any more vertices from the graph to this set without losing connectivity.
  - Every vertex belongs to exactly one connected component.
- A graph G that is not connected has two or more connected components that are disjoint and have G as their union.

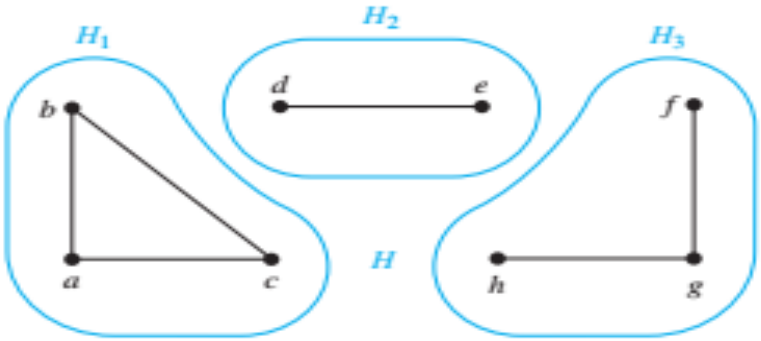


FIGURE 3 The Graph H and Its Connected Components H_1, H_2, and H_3.

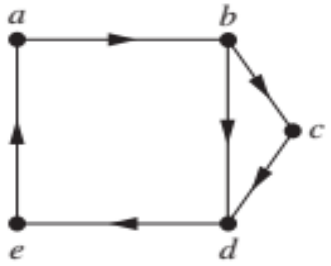
- Graph H looks like a picture of three graphs but is intended to be a picture of one graph. This graph consists of three pieces (subgraphs).
- Each piece by itself is connected, but there are no paths between vertices in different pieces. These connected pieces of a graph are called its *connected components*.

Connectedness in Directed Graphs

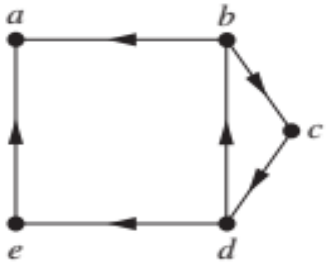
A directed graph is *strongly connected* if there is a path from a to b and from b to a whenever a and b are vertices in the graph.

A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph.

- A directed graph is weakly connected if and only if there is always a path between two vertices when the directions of the edges are disregarded.
- Clearly, any strongly connected directed graph is also weakly connected.
- Are the directed graphs G and H shown below are strongly connected? Are they weakly connected?



G



H

- **Strongly Connected Components (SCC)**
  - **Definition**: A strongly connected component of a directed graph $G = (V, E)$ is a maximal set $v \subseteq V$ such that for each pair of vertices u and v in v, we have $u \rightarrow v$ and $v \rightarrow u$, i.e., vertices u and v are both reachable from each other.
  - The maximal strongly connected subgraphs, are called the *strongly connected components* or *strong components* of G.
  - Note that if a and b are two vertices in a directed graph, their strong components are either the same or disjoint.

Algorithm to Find SCCs: Kosaraju's Algorithm

- Core idea

1. Perform DFS on the original graph and push nodes onto a stack in order of their finishing times (i.e., the node which finished late will be on top of the stack).
2. Transpose the graph (reverse all edges).
3. Pop nodes one by one from the stack and perform DFS on the transposed graph — each DFS visit gives one SCC.

- Algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

1 call DFS(G) to compute finish times $u.f$ for each vertex u
2 create G^T
3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

- Time complexity: $O(V + E)$
- Space complexity: $O(V + E)$

Algorithm to Find SCCs: Tarjan's Algorithm

- Core idea
  - To efficiently identify maximal sets of vertices in a directed graph in a **single DFS traversal** where every vertex is reachable from every other vertex in the same set.
- Steps
  - DFS discovery time (index): each vertex is given a unique discovery **index** as it is first visited during DFS.
  - Low-link value: for each vertex, the algorithm maintains a **low-link value**, which represents the smallest discovery index reachable from that vertex (including itself, descendants, and possible back-edges).
  - Stack to track active vertices: vertices currently in the DFS recursion are pushed onto a stack. This helps in keeping track of the current "search path" of strongly connected components being formed.
  - Component identification:
    - When a vertex's discovery index equals its low-link value, it indicates the root of an SCC.
    - All vertices on the stack up to this root belong to the same SCC, and they are popped together.

Algorithm to Find SCCs: Tarjan's Algorithm

```
algorithm tarjan

input: a directed graph G = (V, E)
output: set of strongly connected components SCCs

index := 0
S := empty stack
SCCs := empty list
for each v in V do
            if v.index is undefined then
                        strongconnect(v)

function strongconnect(v)
            // Set the depth index for v to the smallest unused index
            v.index := index
            v.lowlink := index
            index := index + 1
            S.push(v)
            v.onStack := true

            // Consider successors of v
            for each (v, w) in E do
                        if w.index is undefined then
                        // Successor w has not yet been visited; recurse on it
                                    strongconnect(w)
                                    v.lowlink := min(v.lowlink, w.lowlink)
                        else if w.onStack then
                                    v.lowlink := min(v.lowlink, w.index)
                        // If v is a root node, pop the stack and generate an SCC
            if v.lowlink == v.index then
                        start a new SCC
                        repeat
                                    w := S.pop()
                                    w.onStack := false
                                    add w to current SCC
                        until w = v
            Add current SCC to SCCs
```

Algorithm to Find SCCs: Tarjan's Algorithm

- Time complexity
  - The ***tarjan*** procedure is called once for each node
  - The outer for-loop considers each edge at most once
  - The algorithm's running time is linear in the number of edges and vertices in G, i.e., $O(|V| + |E|)$.
  - To achieve this, the test for "is a vertex w is on the stack" should be done in the constant time – *use of a flag variable for every node (OnStack).*

- Space complexity
  - The ***tarjan*** procedure needs to the maintain the following data for every vertex:
    - One word each for *v.index, v.lowlink*
    - *onStack (1-bit)*
    - *1-bit to determine if index is undefined*
  - Within a stack frame data to be maintained (one word each): node v and *its* current position *in the vertex list (index variable)*
  - The worst-case size of the stack must be $O(|V|)$
  - The size of the edge list must be $\Theta(|E|)$
  - This gives the final analysis of $O(|V|.(2 + 4w)) + \Theta(|E|)$, *where w is machine word size.*